SISTEMA NATYS

Informe de Pruebas de Software

Módulo: Moneda

Integrante	Elias Arma Moises Pereira Edgmairys Aponte Ismael Torres Santiago Loyo
Equipo	Equipo 8
Docente	Profa. Angelismar Terán
Sección	IN3103
Fecha	17-10-2026

1. Introducción

El presente informe documenta el proceso integral de aseguramiento de calidad (QA) y pruebas de software aplicadas a cada uno de los módulos que integran el **Sistema Naty's**. El objetivo principal es verificar y validar el correcto funcionamiento de los flujos de trabajo del sistema, incluyendo la gestión completa del ciclo de vida de los datos (registro, consulta, modificación, eliminación, aprobación y cancelación de acciones).

Para garantizar un nivel óptimo de estabilidad, rendimiento y seguridad, se implementó una estrategia de evaluación multinivel que abarca las siguientes tipologías de pruebas:

- **Pruebas Unitarias y de Humo:** Ejecutadas directamente sobre la capa de modelo mediante **PHPUnit** para verificar el comportamiento aislado de las funciones core y asegurar la estabilidad básica antes de fases avanzadas.
- **Pruebas de Integración y Aceptación:** Realizadas con **PHPUnit** y **Codeception**, validando la interacción entre componentes y garantizando que los módulos cumplan con los criterios de aceptación y requerimientos del usuario.
- **Pruebas de Backend y Seguridad:** Implementadas con **Postman** para auditar el comportamiento de las APIs, el manejo de respuestas HTTP y la resistencia ante vulnerabilidades en los endpoints.
- **Pruebas de Interfaz de Usuario (UI):** Automatizadas con **Selenium** para simular la navegación e interacción directa del usuario en el navegador.
- **Pruebas de Rendimiento y Carga:** Llevadas a cabo mediante **Apache JMeter**, evaluando los tiempos de respuesta y la estabilidad del sistema bajo volúmenes de concurrencia y carga de datos.

Modulo: Moneda

El módulo Moneda se verificó con seis pruebas unitarias (UT-01 a UT-06) que cubren la validación de acciones y datos y el CRUD completo mediante mocks, de modo que se ejecutan sin servidor ni base de datos. Finalmente, una prueba de integración (IT-01) comprobó que el modelo consulta y devuelve monedas reales desde la base de datos MySQL.

1.1 Resumen de resultados

ID	Caso de prueba	Acción	Estado
UT-01	Acción inválida retorna error de estructura	manejarAccion	**Aprobado**
UT-02	Registrar moneda con datos vacíos	registrar	**Aprobado**
UT-03	Registrar moneda exitosamente con mock	registrar	**Aprobado**
UT-04	Obtener moneda exitosamente con mock	obtener	**Aprobado**

ID	Caso de prueba	Acción	Estado
UT-05	Modificar moneda exitosamente con mock	modificar	**Aprobado**
UT-06	Eliminar moneda exitosamente con mock	eliminar	**Aprobado**
Suite	Archivo completo tests/unitarias/MonedaTest.php (6 pruebas unitarias)	todas	**6 de 6 Aprobado**
IT-01	Integración: consultar monedas en la base de datos real	consultar	**Aprobado**
API-01	Backend: consultar las monedas registradas (Postman)	obtenerMonedas	**Aprobado**
SM-01	Humo: el módulo de Moneda inicializa y carga correctamente	todas	**Aprobado**
UI-01	Interfaz y sistema: el formulario Modificar muestra los datos de la moneda (Selenium IDE)	modificar	**Aprobado**
SEC-01	Seguridad: obtener una moneda sin sesión (Postman)	obtener	**Aprobado**
SEC-02	Seguridad: consultar el módulo de monedas sin sesión (Postman)	consultar	**Aprobado**
SEC-03	Seguridad: inyección SQL en el identificador de la moneda (Postman)	obtener	**Aprobado**
RD-01	Rendimiento: 50 usuarios consultan el listado de monedas simultáneamente (JMeter)	login / consultar	**Aprobado**
AC-01	Aceptación: consultar el panel de monedas (Codeception)	consultar	**Aprobado**

1.2 Cobertura del CRUD

Operación	Prueba que la cubre	Comportamiento verificado
Validación de acciones	UT-01 · Acción inválida	**Rechaza comandos no reconocidos devolviendo status = false y el mensaje «Accion Invalida.».**
Validación de datos	UT-02 · Registrar moneda con datos vacíos	**El validador detiene el registro cuando el nombre está vacío y devuelve status = false.**
Crear / Registrar	UT-03 · Registrar moneda mediante mock	**Inserta la moneda con nombre y descripción válidos y confirma la ejecución del INSERT.**
Leer	UT-04 · Obtener moneda mediante mock	**Recupera la moneda y sus datos (id, nombre, descripción y estado) desde la consulta simulada.**
Actualizar	UT-05 · Modificar moneda mediante mock	Actualiza el nombre y la descripción de la moneda y confirma la transacción.
Eliminar	UT-06 · Eliminar moneda mediante mock	Desactiva la moneda y

Operación	Prueba que la cubre	Comportamiento verificado
		confirma la transacción.

1.3 Prueba unitaria UT-01: Acción inválida retorna error de estructura

Campo	Detalle
ID	UT-01
Nombre	Acción inválida retorna error de estructura
Tipo de prueba	Unitaria
Herramienta	PHPUnit 11.5.57
Archivo y método	tests/unitarias/MonedaTest.php · test_accion_invalida_retorna_error
Descripción	Verifica que el método centralizador manejarAccion rechace cualquier acción no reconocida ('accion_falsa'), devolviendo un arreglo con status = false y el mensaje «Accion Invalida.» sin afectar la base de datos.
Precondiciones	PHPUnit instalado en el proyecto · Configuración phpunit.xml presente · Entorno aislado sin requerir servidor MySQL activo.
Datos de entrada	**Acción: accion_falsa / JSON: {}**
Pasos	1. Se abrió la terminal CMD en la ruta del proyecto C:\xampp\htdocs\larence. 2. Se verificó PHPUnit con: vendor\bin\phpunit --version 3. Se ejecutó la prueba unitaria específica: vendor\bin\phpunit tests\unitarias\ MonedaTest.php --filter test_accion_invalida_retorna_error
Resultado esperado	El modelo procesa la acción desconocida, devuelve un arreglo con status = false y el mensaje de error «Accion Invalida.».
Resultado obtenido	El modelo devolvió correctamente status = false y el mensaje esperado. PHPUnit validó satisfactoriamente todas las aserciones de la estructura de error.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

1.3.1 Evidencias

Captura 1 – Código del método aislado

```php
/** @test */
public function test_accion_invalida_retorna_error(): void {
    $moneda = new Moneda();
    $resultado = $moneda->manejarAccion('accion_falsa', '{}');

    $this->assertIsArray($resultado);
    $this->assertFalse($resultado['status']);
    $this->assertEquals('Accion Invalida.', $resultado['msj']);
}
```

Captura 2 – Ejecución de UT-01

```
C:\xampp\htdocs\larence>vendor\bin\phpunit tests\unitarias\MonedaTest.php --filter test_accion_invalida_retorna_error
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\larence\phpunit.xml

.                                                                   1 / 1 (100%)

Time: 00:00.109, Memory: 8.00 MB

OK, but there were issues!
Tests: 1, Assertions: 3, PHPUnit Deprecations: 6.
```

Prueba aprobada con XAMPP apagado

2.1 Prueba unitaria UT-02: Registrar moneda con datos vacíos

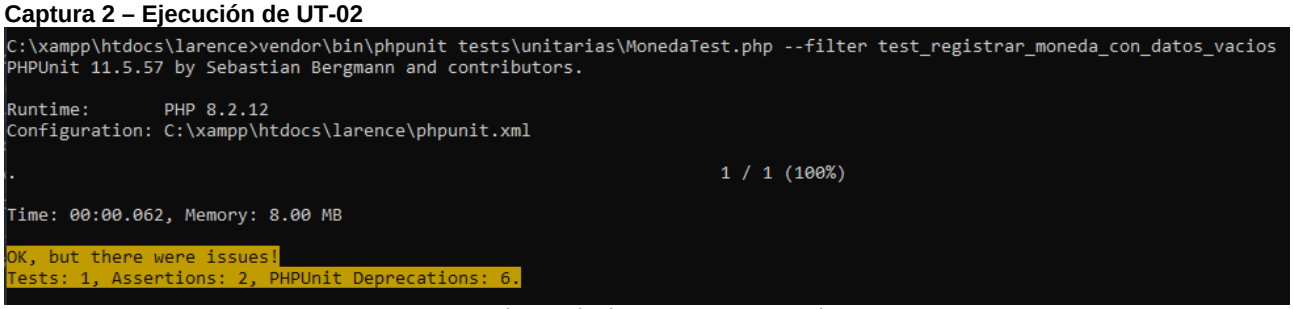
ID	UT-02
Nombre	Registrar moneda con datos vacíos
Tipo de prueba	Unitaria
Acción evaluada	registrar
Archivo y método	tests/unitarias/MonedaTest.php · test_registrar_moneda_con_datos_vacios
Descripción	Verifica que el método registrar rechace una solicitud con el nombre vacío, deteniendo el proceso antes de insertar la moneda y devolviendo status = false.
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · PHPUnit configurado y operativo en el proyecto.
Simulación (mock)	No requiere simulación de PDO: el validador de datos detiene la operación antes de cualquier acceso a la base de datos.
Datos de entrada	**`Acción: registrar / JSON: {"nombre": ""}`**
Pasos	1. Se aseguró la inclusión del método de prueba en tests/unitarias/MonedaTest.php. 2. Se detuvieron los servicios de Apache y MySQL en el panel de XAMPP para garantizar el aislamiento. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\unitarias\MonedaTest.php --filter test_registrar_moneda_con_datos_vacios --testdox 4. Se registró la salida obtenida en consola.
Resultado esperado	El validador detecta el nombre vacío, no registra la moneda y devuelve status = false.
Resultado obtenido	✔️ Registrar moneda con datos vacios. 1 prueba ejecutada con las aserciones correctas, sin fallos. El modelo rechazó la solicitud incompleta sin acceder a la base de datos.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código del método aislado

```php
/** @test */
public function test_registrar_moneda_con_datos_vacios(): void {
    $moneda = new Moneda();
    $resultado = $moneda->manejarAccion('registrar', json_encode(['nombre' => '']));

    $this->assertIsArray($resultado);
    $this->assertFalse($resultado['status']);
}
```

Captura 2 – Ejecución de UT-02



Prueba aprobada con XAMPP apagado

2.2 Prueba unitaria UT-03: Registrar moneda exitosamente con mock

ID	UT-03
Nombre	Registrar moneda exitosamente con mock
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	registrar
Archivo y método	tests/unitarias/MonedaTest.php · test_registrar_moneda_exitosamente_con_mock
Descripción	Verifica que el método registrar inserte una moneda con nombre y descripción válidos mediante una conexión PDO simulada, retornando status = true y el mensaje «Moneda registrada con exito.».
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · MonedaTest.php configurado con mocks de PDO y PDOStatement.
Simulación (mock)	Se simuló la conexión PDO y la sentencia PDOStatement para interceptar prepare y execute, confirmando la ejecución del INSERT sobre la tabla monedas.
Datos de entrada	**`Acción: registrar / JSON: {"nombre": "Dolar", "descripcion": "Dolar Estadounidense"}`**
Pasos	1. Se aseguró la inclusión del método de prueba en tests/unitarias/MonedaTest.php. 2. Se detuvieron los servicios de Apache y MySQL en el panel de XAMPP para garantizar el aislamiento. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\unitarias\MonedaTest.php --filter test_registrar_moneda_exitosamente_con_mock --testdox 4. Se registró la salida obtenida en consola.
Resultado esperado	El modelo valida los datos, registra la moneda y devuelve status = true con el mensaje «Moneda registrada con exito.».
Resultado obtenido	✔️ Registrar moneda exitosamente con mock. 1 prueba ejecutada con las aserciones correctas, sin fallos. El modelo registró la moneda y confirmó la operación simulada.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código del método aislado

```php
/** @test */
public function test_registrar_moneda_exitosamente_con_mock(): void {
    $stmtMock = $this->createMock(PDOStatement::class);
    $stmtMock->expects($this->once())->method('execute')->willReturn(true);

    $pdoMock = $this->createMock(PDO::class);
    $pdoMock->expects($this->once())->method('prepare')->willReturn($stmtMock);

    $monedaMock = $this->getMockBuilder(Moneda::class)
                       ->onlyMethods(['getConnectionNegocio', 'closeConnection'])
                       ->getMock();

    $monedaMock->method('getConnectionNegocio')->willReturn($pdoMock);
    $monedaMock->method('closeConnection')->willReturn(null);

    // Sin tildes para superar la expresión regular del trait de validación
    $jsonDatos = json_encode([
        'nombre'      => 'Dolar',
        'descripcion' => 'Dolar Estadounidense'
    ]);

    $resultado = $monedaMock->manejarAccion('registrar', $jsonDatos);

    $this->assertIsArray($resultado);
    $this->assertTrue($resultado['status'], 'Error: ' . ($resultado['msj'] ?? ''));
    $this->assertEquals('Moneda registrada con exito.', $resultado['msj']);
}
```

Captura 2 – Ejecución de UT-03

```
C:\xampp\htdocs\larence>vendor\bin\phpunit tests\unitarias\MonedaTest.php --filter test_registrar_moneda_exitosamente_con_mock
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\larence\phpunit.xml

.                                                                   1 / 1 (100%)

Time: 00:00.130, Memory: 8.00 MB

OK, but there were issues!
Tests: 1, Assertions: 5, PHPUnit Deprecations: 6.
```

Prueba aprobada con XAMPP apagado

2.3 Prueba unitaria UT-04: Obtener moneda exitosamente con mock

ID	UT-04
Nombre	Obtener moneda exitosamente con mock
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	obtener
Archivo y método	tests/unitarias/MonedaTest.php · test_obtener_moneda_exitosamente_con_mock
Descripción	Verifica que el método obtener recupere una moneda existente mediante una consulta PDO simulada, retornando status = true, el mensaje «Moneda encontrada con exito.» y los datos de la moneda (id_moneda, nombre, descripción y estado).
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · MonedaTest.php configurado con mocks de PDO y PDOStatement.
Simulación (mock)	Se simuló la conexión PDO y la sentencia PDOStatement para interceptar prepare, execute y fetch, devolviendo una moneda con id_moneda = 1, nombre_moneda = «Dolar», descripcion_moneda = «Dolar estadounidense» y status = 1.

Datos de entrada	**Acción: obtener / JSON: {"id": 1, "nombre": "Dolar", "descripcion": "Dolar estadounidense"}**
Pasos	1. Se aseguró la inclusión del método de prueba en tests/unitarias/MonedaTest.php. 2. Se detuvieron los servicios de Apache y MySQL en el panel de XAMPP para garantizar el aislamiento. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\unitarias\MonedaTest.php --filter test_obtener_moneda_exitosamente_con_mock --testdox 4. Se registró la salida obtenida en consola.
Resultado esperado	El modelo devuelve status = true, el mensaje «Moneda encontrada con exito.» y los datos de la moneda consultada (id_moneda = 1) sin requerir conexión física a la base de datos.
Resultado obtenido	✔️ Obtener moneda exitosamente con mock. 1 prueba ejecutada con las aserciones correctas, sin fallos. El modelo devolvió los datos esperados mediante el aislamiento con mocks.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código del método aislado

```php
/** @test */
public function test_obtener_moneda_exitosamente_con_mock(): void {
    $stmtMock = $this->createMock(PDOStatement::class);
    $stmtMock->expects($this->once())->method('execute')->willReturn(true);
    $stmtMock->expects($this->once())->method('fetch')->willReturn([
        'id_moneda'          => 1,
        'nombre_moneda'      => 'Dolar',
        'descripcion_moneda' => 'Dolar estadounidense',
        'status'             => 1
    ]);

    $pdoMock = $this->createMock(PDO::class);
    $pdoMock->expects($this->once())->method('prepare')->willReturn($stmtMock);

    $monedaMock = $this->getMockBuilder(Moneda::class)
                       ->onlyMethods(['getConnectionNegocio', 'closeConnection'])
                       ->getMock();

    $monedaMock->method('getConnectionNegocio')->willReturn($pdoMock);
    $monedaMock->method('closeConnection')->willReturn(null);

    $jsonDatos = json_encode([
        'id'          => 1,
        'nombre'      => 'Dolar',
        'descripcion' => 'Dolar estadounidense'
    ]);

    $resultado = $monedaMock->manejarAccion('obtener', $jsonDatos);

    $this->assertIsArray($resultado);
    $this->assertTrue($resultado['status'], 'Error: ' . ($resultado['msj'] ?? ''));
    $this->assertEquals('Moneda encontrada con exito.', $resultado['msj']);
    $this->assertEquals(1, $resultado['data']['id_moneda']);
}
```

Captura 2 – Ejecución de UT-04



Prueba aprobada con XAMPP apagado

2.4 Prueba unitaria UT-05: Modificar moneda exitosamente con mock

ID	UT-05
Nombre	Modificar moneda exitosamente con mock
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	modificar
Archivo y método	tests/unitarias/MonedaTest.php · test_modificar_moneda_exitosamente_con_mock
Descripción	Verifica que el método modificar actualice el nombre y la descripción de una moneda existente y confirme la transacción, devolviendo status = true y el mensaje «Moneda actualizada con exito.» sin requerir base de datos física.
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · MonedaTest.php configurado con mocks de PDO y PDOStatement.
Simulación (mock)	Se simuló la conexión PDO y las sentencias PDOStatement para interceptar la consulta de la moneda (SELECT) y la actualización (UPDATE monedas), confirmando la ejecución de la transacción.
Datos de entrada	**`Acción: modificar / JSON: {"id": 1, "nombre": "Dolar Editado", "descripcion": "USD Actualizado"}`**
Pasos	1. Se aseguró la inclusión del método de prueba en tests/unitarias/MonedaTest.php. 2. Se detuvieron los servicios de Apache y MySQL en el panel de XAMPP para garantizar el aislamiento. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\unitarias\MonedaTest.php --filter test_modificar_moneda_exitosamente_con_mock --testdox 4. Se registró la salida obtenida en consola.
Resultado esperado	El modelo procesa la actualización, confirma la transacción y devuelve status = true con el mensaje «Moneda actualizada con exito.».
Resultado obtenido	✔️ Modificar moneda exitosamente con mock. 1 prueba ejecutada con las aserciones correctas, sin fallos. El modelo actualizó la moneda y confirmó la transacción simulada.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código del método aislado

```php
/** @test */
public function test_modificar_moneda_exitosamente_con_mock(): void {
    $stmtSelect = $this->createMock(PDOStatement::class);
    $stmtSelect->method('execute')->willReturn(true);
    $stmtSelect->method('fetch')->willReturn(['id_moneda' => 1, 'nombre_moneda' => 'Dolar']);

    $stmtUpdate = $this->createMock(PDOStatement::class);
    $stmtUpdate->method('execute')->willReturn(true);

    $pdoMock = $this->createMock(PDO::class);
    $pdoMock->method('prepare')->willReturnCallback(function($sql) use ($stmtSelect, $stmtUpdate) {
        if (strpos($sql, 'UPDATE monedas') !== false) return $stmtUpdate;
        return $stmtSelect;
    });

    $monedaMock = $this->getMockBuilder(Moneda::class)
                       ->onlyMethods(['getConnectionNegocio', 'closeConnection'])
                       ->getMock();

    $monedaMock->method('getConnectionNegocio')->willReturn($pdoMock);
    $monedaMock->method('closeConnection')->willReturn(null);

    $jsonDatos = json_encode([
        'id'          => 1,
        'nombre'      => 'Dolar Editado',
        'descripcion' => 'USD Actualizado'
    ]);

    $resultado = $monedaMock->manejarAccion('modificar', $jsonDatos);

    $this->assertIsArray($resultado);
    $this->assertTrue($resultado['status'], 'Error: ' . ($resultado['msj'] ?? ''));
    $this->assertEquals('Moneda actualizada con exito.', $resultado['msj']);
}
```

Captura 2 – Ejecución de UT-05

```
C:\xampp\htdocs\larence>vendor\bin\phpunit tests\unitarias\MonedaTest.php --filter test_modificar_moneda_exitosamente_con_mock
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\larence\phpunit.xml

.                                                                   1 / 1 (100%)

Time: 00:00.097, Memory: 10.00 MB

OK, but there were issues!
Tests: 1, Assertions: 3, PHPUnit Deprecations: 6.
```

Prueba aprobada con XAMPP apagado

2.5 Prueba unitaria UT-06: Eliminar moneda exitosamente con mock

ID	UT-06
Nombre	Eliminar moneda exitosamente con mock
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	eliminar
Archivo y método	tests/unitarias/MonedaTest.php · test_eliminar_moneda_exitosamente_con_mock
Descripción	Verifica que al ejecutar la acción eliminar con una moneda válida, el modelo la desactive y confirme la transacción, devolviendo status = true y el mensaje «Moneda eliminada con exito.» sin requerir base de datos física.

Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · MonedaTest.php configurado con mocks de PDO y PDOStatement.
Simulación (mock)	Se simuló la conexión PDO para interceptar la consulta de la moneda (SELECT) y la desactivación (UPDATE monedas), confirmando la ejecución de la transacción.
Datos de entrada	**`Acción: eliminar / JSON: {"id": 1, "nombre": "Dolar", "descripcion": "Dolar"}`**
Pasos	1. Se aseguró la inclusión del método de prueba en tests/unitarias/MonedaTest.php. 2. Se detuvieron los servicios de Apache y MySQL en el panel de XAMPP para garantizar el aislamiento. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\unitarias\MonedaTest.php --filter test_eliminar_moneda_exitosamente_con_mock --testdox 4. Se registró la salida obtenida en consola.
Resultado esperado	El modelo desactiva la moneda, confirma la transacción y devuelve status = true con el mensaje «Moneda eliminada con exito.».
Resultado obtenido	✔️ Eliminar moneda exitosamente con mock. 1 prueba ejecutada con las aserciones correctas, sin fallos. El modelo confirmó la transacción simulada y desactivó la moneda correctamente.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código del método aislado

```php
/** @test */
public function test_eliminar_moneda_exitosamente_con_mock(): void {
    $stmtSelect = $this->createMock(PDOStatement::class);
    $stmtSelect->method('execute')->willReturn(true);
    $stmtSelect->method('fetch')->willReturn(['id_moneda' => 1, 'nombre_moneda' => 'Dolar']);

    $stmtDelete = $this->createMock(PDOStatement::class);
    $stmtDelete->method('execute')->willReturn(true);

    $pdoMock = $this->createMock(PDO::class);
    $pdoMock->method('prepare')->willReturnCallback(function($sql) use ($stmtSelect, $stmtDelete) {
        if (strpos($sql, 'UPDATE monedas') !== false) return $stmtDelete;
        return $stmtSelect;
    });

    $monedaMock = $this->getMockBuilder(Moneda::class)
                       ->onlyMethods(['getConnectionNegocio', 'closeConnection'])
                       ->getMock();

    $monedaMock->method('getConnectionNegocio')->willReturn($pdoMock);
    $monedaMock->method('closeConnection')->willReturn(null);

    $jsonDatos = json_encode([
        'id'          => 1,
        'nombre'      => 'Dolar',
        'descripcion' => 'Dolar'
    ]);

    $resultado = $monedaMock->manejarAccion('eliminar', $jsonDatos);

    $this->assertIsArray($resultado);
    $this->assertTrue($resultado['status'], 'Error: ' . ($resultado['msj'] ?? ''));
    $this->assertEquals('Moneda eliminada con exito.', $resultado['msj']);
}
```

Captura 2 – Ejecución de UT-06

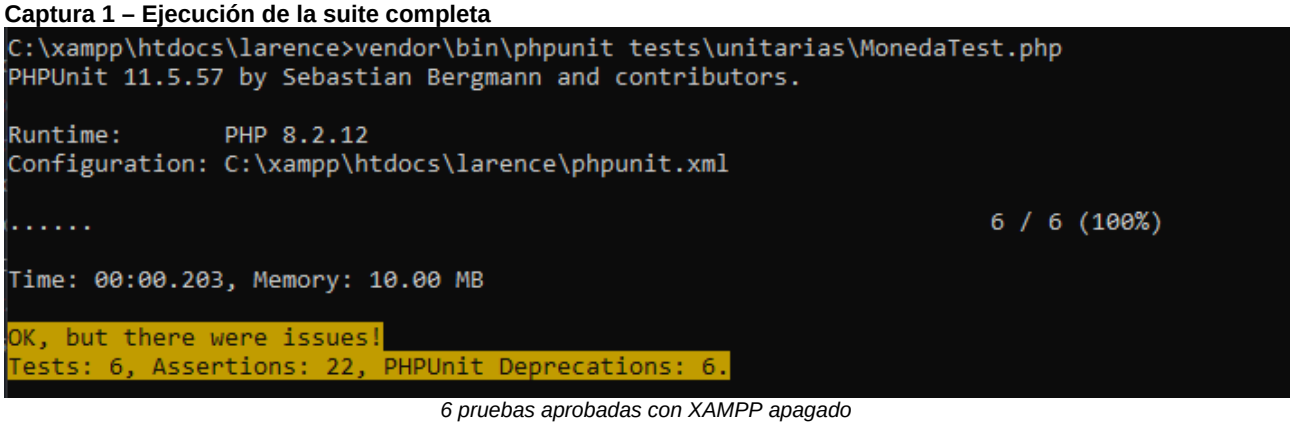


Prueba aprobada con XAMPP apagado

2.6 Ejecución completa de la suite

Comando	vendor\bin\phpunit tests\unitarias\MonedaTest.php --testdox
Precondiciones	XAMPP apagado (Apache y MySQL detenidos).
Resultado esperado	Las 6 pruebas del modelo Moneda aprobadas sin conexión a la base de datos.
Resultado obtenido	6 de 6 pruebas aprobadas, 22 aserciones, sin fallos (5,254 s; 10 MB). PHPUnit mostró 6 avisos «PHPUnit Deprecations» por el uso de la anotación /** @test */ en comentarios, un estilo que PHPUnit 11 marca como obsoleto. No afectan los resultados.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Ejecución de la suite completa



6 pruebas aprobadas con XAMPP apagado

3.1 Prueba de integración IT-01

A diferencia de las pruebas unitarias, la prueba de integración usa el modelo Moneda real conectado a la base de datos MySQL, para comprobar que el código y la base de datos trabajan correctamente en conjunto y que el listado de monedas coincide con los registros persistidos.

ID	IT-01
Nombre	Integración: consultar monedas en la base de datos real
Tipo de prueba	Integración (Modelo Moneda + MySQL)
Herramienta	PHPUnit 11.5.57
Archivo y método	tests/integracion/MonedaIntegracionTest.php · test_consultar_monedas_en_la_base_real
Descripción	Instancia el modelo Moneda real, ejecuta la acción consultar y compara el primer registro devuelto con la tabla monedas (status = 1) en MySQL, comprobando que la respuesta del modelo coincide con la base de datos.

Precondiciones	MySQL activo en XAMPP · Base de datos larence con al menos una moneda activa.
Datos de entrada	**`Acción: consultar / JSON: {}`**
Pasos	1. Se encendió y verificó el servicio MySQL en el panel de control de XAMPP. 2. Se creó y configuró el archivo tests/integracion/MonedaIntegracionTest.php. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\integracion\ MonedaIntegracionTest.php --testdox 4. Se registró la salida exitosa obtenida en consola.
Resultado esperado	El modelo devuelve status = true con el listado de monedas y el primer id_moneda coincide con un registro activo real de la tabla monedas; si no hay registros, devuelve el mensaje «Monedas no encontradas o inactivas».
Resultado obtenido	1 / 1 prueba ejecutada con éxito. OK (1 test, 3 aserciones). 2,413 s; 8 MB. El modelo respondió correctamente y la consulta directa a MySQL confirmó que la moneda devuelta existe en la base de datos real.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código de la prueba de integración

```php
public function test_consultar_monedas_en_la_base_real(): void {
    $monedaModelo = new Moneda();
    $resultado = $monedaModelo->manejarAccion('consultar', '{}');

    $this->assertIsArray($resultado);
    if ($resultado['status']) {
        $this->assertNotEmpty($resultado['data']);
        $idMoneda = (int)$resultado['data'][0]['id_moneda'];
        $monedaDb = $this->db->query("SELECT id_moneda FROM monedas WHERE id_moneda = $idMoneda AND status = 1")->fetch(PDO::FETCH_ASSOC);
        $this->assertNotFalse($monedaDb);
    } else {
        $this->assertEquals('Monedas no encontradas o inactivas', $resultado['msj']);
    }
}
```

Captura 2 – Resultado obtenido de IT-01

```
C:\xampp\htdocs\larence>vendor\bin\phpunit tests\integracion\MonedaIntegracionTest.php
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\larence\phpunit.xml

.                                                                   1 / 1 (100%)

Time: 00:00.282, Memory: 8.00 MB

OK (1 test, 3 assertions)
```

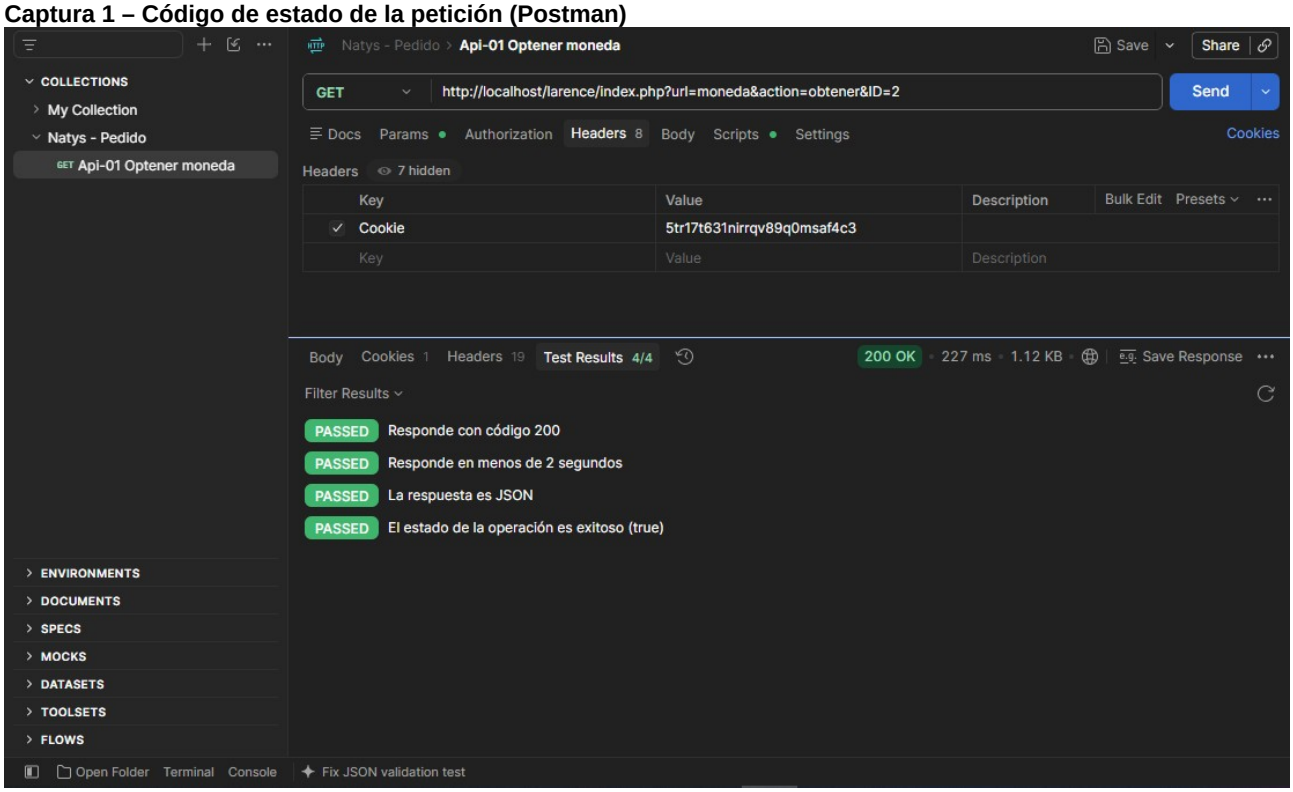
Resultado de la prueba de integración

4.1 Prueba de backend (API) API-01: consultar las monedas registradas

La prueba de backend se comunica directamente con el controlador mediante peticiones HTTP, sin usar la interfaz web, para verificar que el endpoint del módulo Moneda devuelva el código de estado y el JSON esperados.

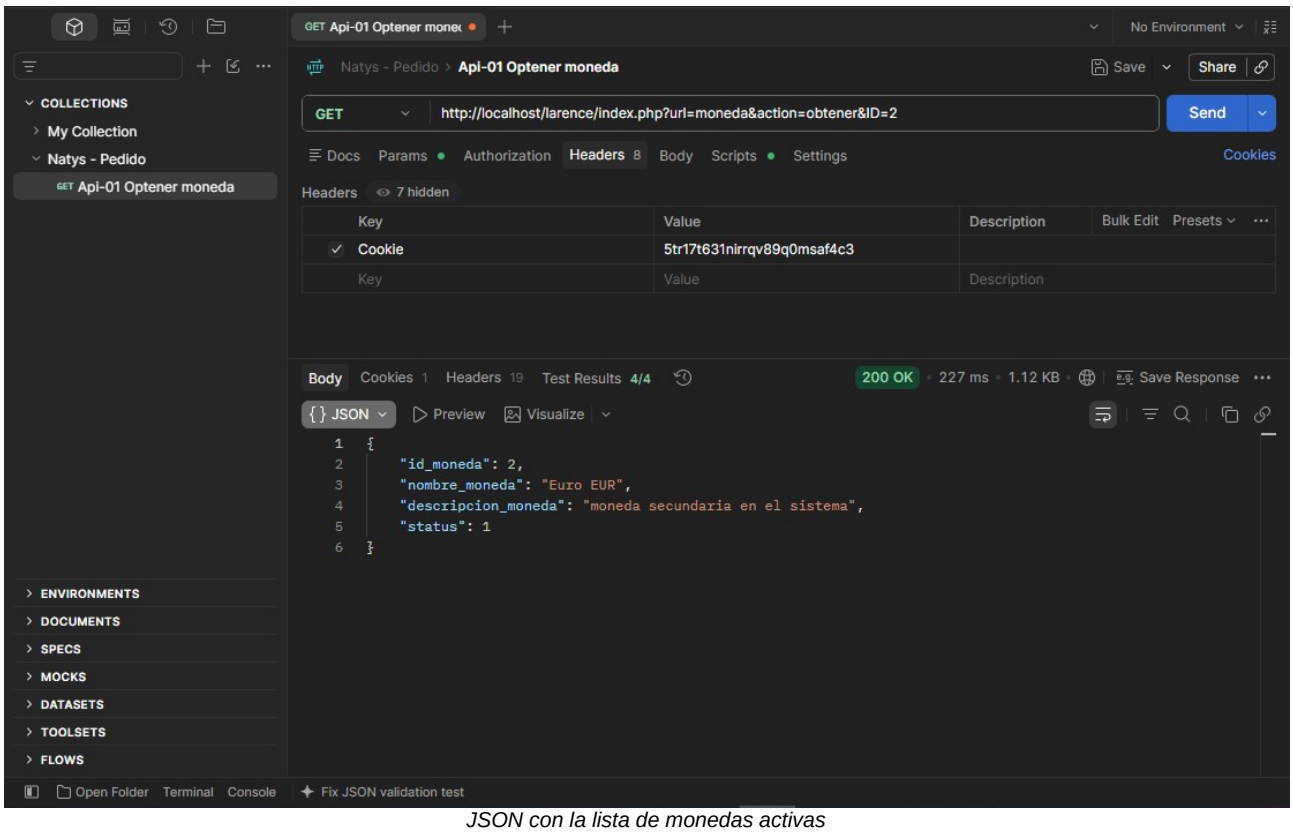
ID	API-01
Nombre	Consultar las monedas registradas
Tipo de prueba	Backend (API)
Herramienta	Postman · colección «Natys – Moneda»
Petición	`GET http://localhost/larence/index.php?url=moneda&action=obtenerMonedas`
Descripción	Verifica que el endpoint obtenerMonedas del MonedaController devuelva la lista de monedas activas en formato JSON.
Precondiciones	Apache y MySQL activos en XAMPP · sesión iniciada en Natys · cookie de sesión PHPSESSID copiada desde el navegador.
Datos de entrada	`Encabezado: Cookie: PHPSESSID=<sesión activa>`
Pasos	1. Se inició sesión en Natys y se copió la cookie PHPSESSID desde las herramientas del navegador (Application → Cookies). 2. En Postman se creó la colección «Natys – Moneda» y la petición GET «API-01 Consultar monedas». 3. Se agregó el encabezado Cookie con el valor PHPSESSID=<sesión>. 4. Se envió la petición (Send) y se verificó el código de estado. 5. Se revisó la respuesta JSON en la pestaña Body (Pretty).
Resultado esperado	Código 200, respuesta JSON en menos de 2 segundos, con la lista de monedas activas y sus campos.
Resultado obtenido	Código 200 OK; respuesta JSON válida con la lista de monedas activas. La consola de Postman muestra una única petición GET con estado 200, sin redirecciones.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código de estado de la petición (Postman)



200 OK en la consulta de monedas

Captura 2 – Respuesta JSON de las monedas



JSON con la lista de monedas activas

5.1 Prueba de humo SM-01: Moneda

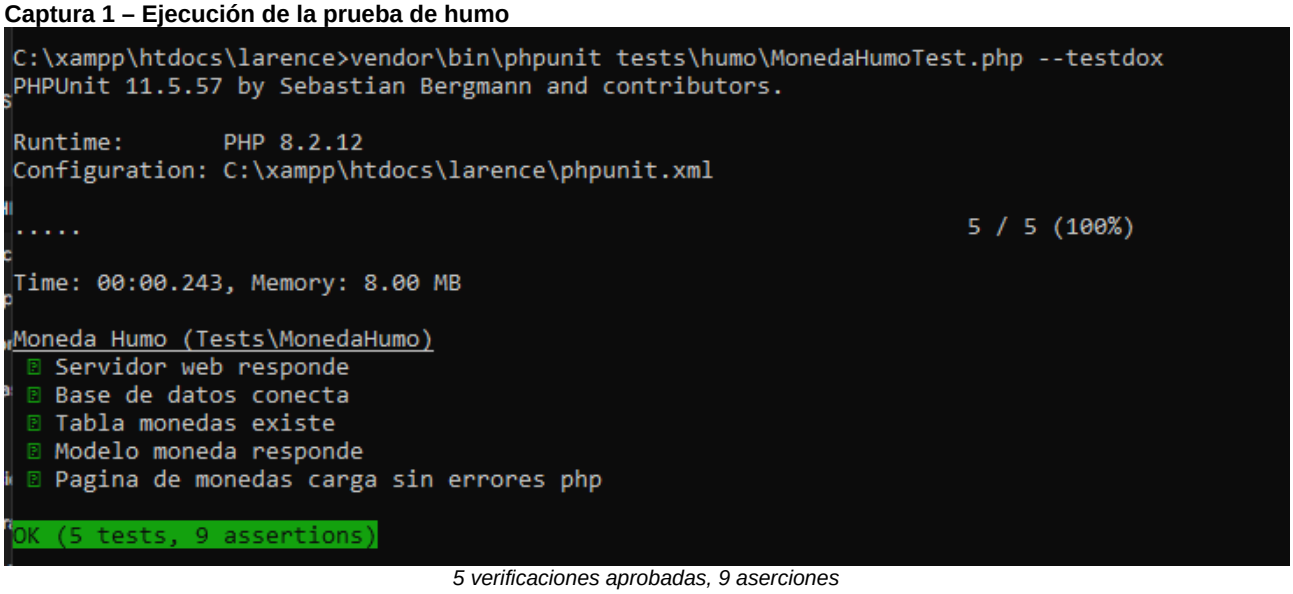
La prueba de humo es una verificación rápida que se ejecuta antes de las pruebas largas para confirmar que el módulo enciende correctamente: el servidor web, la base de datos, la tabla, el modelo y la página responden sin errores.

ID	SM-01
Nombre	El módulo de Moneda enciende correctamente
Tipo de prueba	Humo (smoke test)
Herramienta	PHPUnit 11.5.57
Archivo	tests/humo/MonedaHumoTest.php (5 verificaciones)
Descripción	Comprueba en menos de un segundo que el servidor web responde, que la base de datos conecta, que existe la tabla monedas, que el modelo Moneda responde y que la página del módulo carga sin errores de PHP.
Precondiciones	Apache y MySQL encendidos en XAMPP.
Pasos	1. Se encendieron Apache y MySQL en XAMPP. 2. Se creó el archivo tests/humo/MonedaHumoTest.php. 3. Se ejecutó: vendor\bin\phpunit tests/humo/MonedaHumoTest.php --testdox 4. Se registró la salida de la terminal.
Resultado esperado	Las 5 verificaciones aprobadas en pocos milisegundos.
Resultado obtenido	OK: 5 pruebas y 9 aserciones aprobadas en menos de un segundo.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

#	Verificación	Si fallara, indicaría…	Resultado
1	El servidor web responde (código	Apache apagado	**Aprobado**

	200)		
2	La base de datos conecta (SELECT 1)	MySQL apagado o mal configurado	**Aprobado**
3	Existe la tabla monedas	Base de datos incompleta	**Aprobado**
4	El modelo Moneda responde a consultar	Error grave en el código del modelo	**Aprobado**
5	La página de Monedas carga sin errores de PHP	Fatal error, Parse error o excepción no capturada	**Aprobado**

Captura 1 – Ejecución de la prueba de humo



5 verificaciones aprobadas, 9 aserciones

6.1 Prueba de interfaz y sistema UI-01: Moneda

La prueba de interfaz se ejecuta desde el navegador, como lo haría un usuario real: un robot abre la pantalla de Moneda, presiona los botones y verifica lo que aparece. Al recorrer el módulo sin mirar el código, también funciona como prueba de sistema (caja negra).

ID	UI-01
Nombre	El formulario Modificar muestra los datos de la moneda
Tipo de prueba	Interfaz y sistema / funcional (caja negra)
Herramienta	Selenium IDE (extensión de Firefox)
Descripción	Abre el módulo Monedas, presiona Modificar en la primera moneda y verifica que el formulario se abra con el nombre y la descripción cargados.
Precondiciones	Apache y MySQL activos · sesión iniciada en Natys en Firefox · existe al menos una moneda registrada
Pasos	1. Se instaló Selenium IDE en Firefox y se creó el proyecto «Natys Monedas». 2. Se abrió index.php?url=moneda con la sesión iniciada. 3. Se presionó Modificar en la primera moneda de la tabla. 4. Se verificaron el título del formulario y los campos nombre y descripción.
Resultado esperado	El robot abre el formulario «Modificar Moneda» con el nombre y la descripción de la moneda seleccionada.
Resultado obtenido	Los 8 pasos se ejecutaron en verde y la terminal muestra la prueba aprobada.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

#	Comando	Objetivo	Qué hace

1	open	index.php?url=moneda	**Abre el módulo Monedas**
2	set window size	1888x1096	**Fija el tamaño de la ventana**
3	wait for element visible	css=#add-row tbody tr.odd	**Espera a que la tabla muestre las monedas**
4	click	css=#add-row tbody tr.odd:nth-child(1) .btn:nth-child(2)	**Presiona Modificar en la primera moneda**
5	pause	2000	**Espera la carga de los datos**
6	assert text	id=monedaModalModificarLabel = «Modificar Moneda»	**Verifica el título del formulario**
7	assert element present	id=nombreMonedaEdit	**Verifica el campo del nombre**
8	assert element present	id=descripcionMonedaEdit	**Verifica el campo de la descripción**

Captura 1 – Ejecución de la prueba UI-01 en la terminal (CMD)

```
C:\xampp\htdocs\larence\tests\selenium>python MonedaSelenium.py
Iniciando sesión...
No se detectó ningún tour activo.
Navegando a la sección de Monedas...
No se detectó ningún tour activo.
Abriendo modal de nueva moneda...
Rellenando datos de la moneda...
Guardando moneda...
¡Moneda registrada con éxito!
Presiona ENTER en la terminal para cerrar el navegador...
```

La terminal muestra la prueba ejecutada y aprobada

Captura 2 – Vista del módulo durante la prueba UI-01



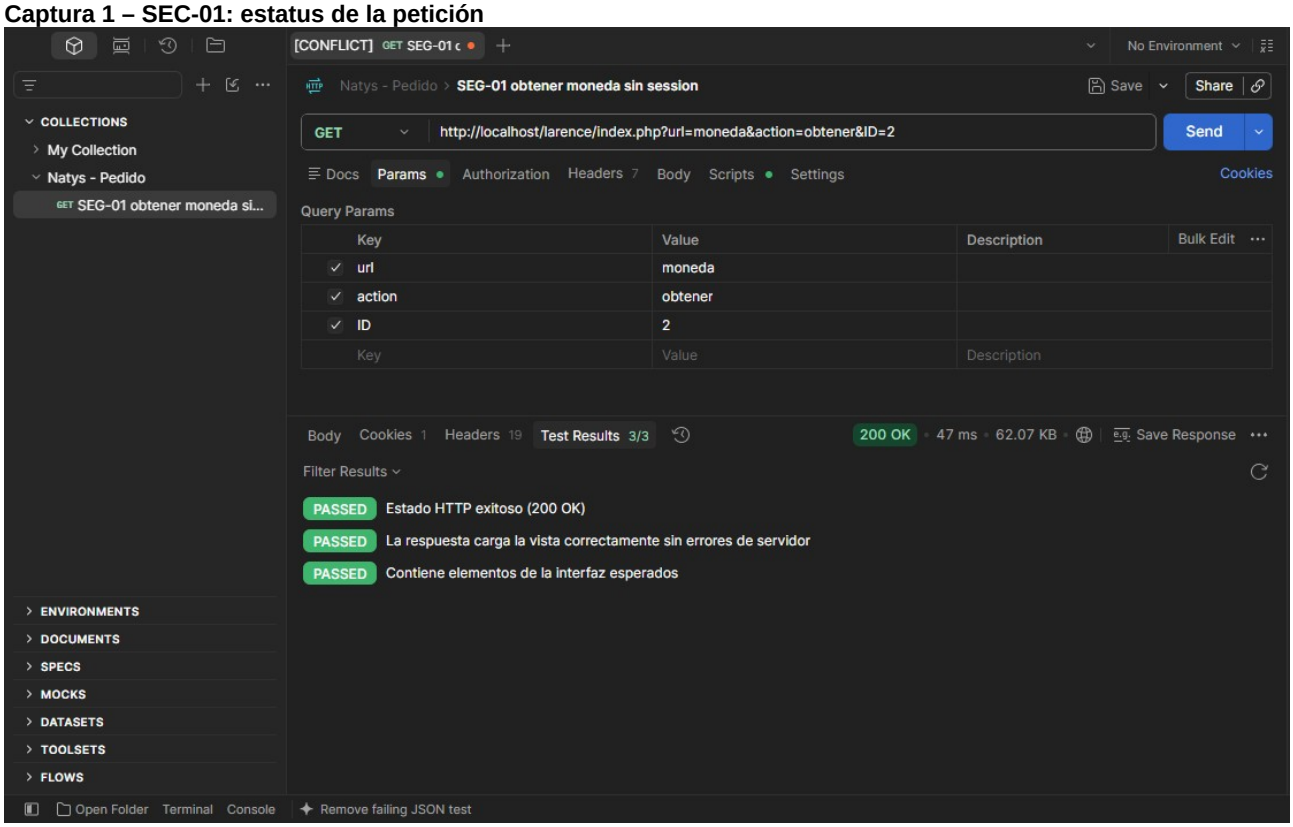
Vista «Gestión de Monedas» con la tabla de monedas registradas

7.1 Prueba de seguridad SEC-01: obtener una moneda sin sesión

La prueba intenta obtener un registro del módulo sin haber iniciado sesión, para comprobar que el sistema no expone datos a usuarios anónimos.

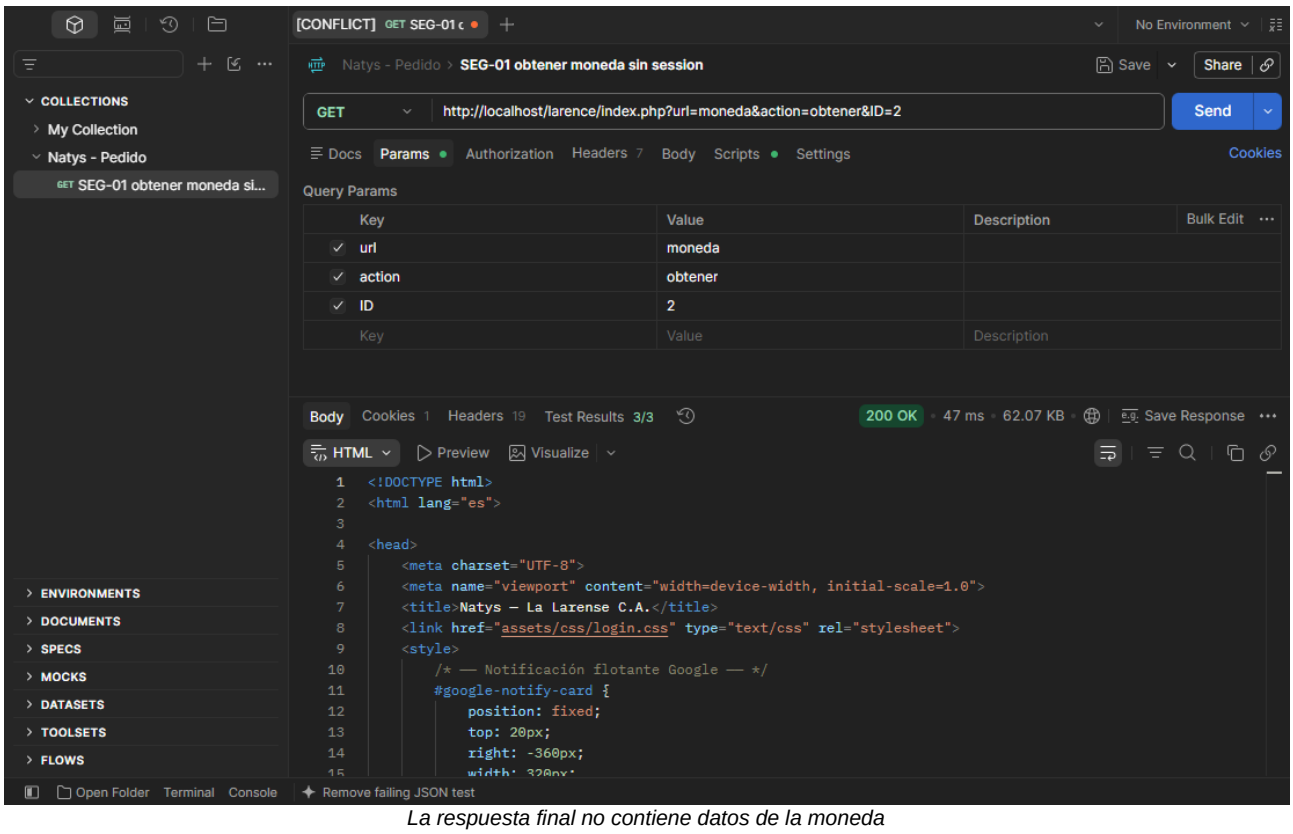
ID	SEC-01
Nombre	Un usuario sin sesión no puede obtener una moneda
Tipo de prueba	Seguridad (control de acceso)
Herramienta	Postman
Petición	`GET http://localhost/larence/index.php?url=moneda&action=obtener&ID=2`
Precondiciones	Apache y MySQL activos · petición sin una sesión autenticada · redirecciones seguidas por Postman
Pasos	1. Se duplicó la petición API-01 y se renombró como SEC-01. 2. Se eliminó la cookie de autenticación de la petición. 3. Se envió la petición GET y se siguió la redirección. 4. Se registraron el código final y la respuesta HTML del servidor.
Resultado esperado	El servidor redirige al inicio de sesión y la respuesta final no incluye datos de la moneda.
Resultado obtenido	200 OK en 47 ms tras seguir la redirección. La respuesta final es la página de inicio de sesión y no contiene datos de la moneda. Las 3 aserciones de Postman se aprobaron.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – SEC-01: estatus de la petición



200 OK en 47 ms y respuesta HTML del inicio de sesión

Captura 2 – SEC-01: respuesta del servidor (HTML)



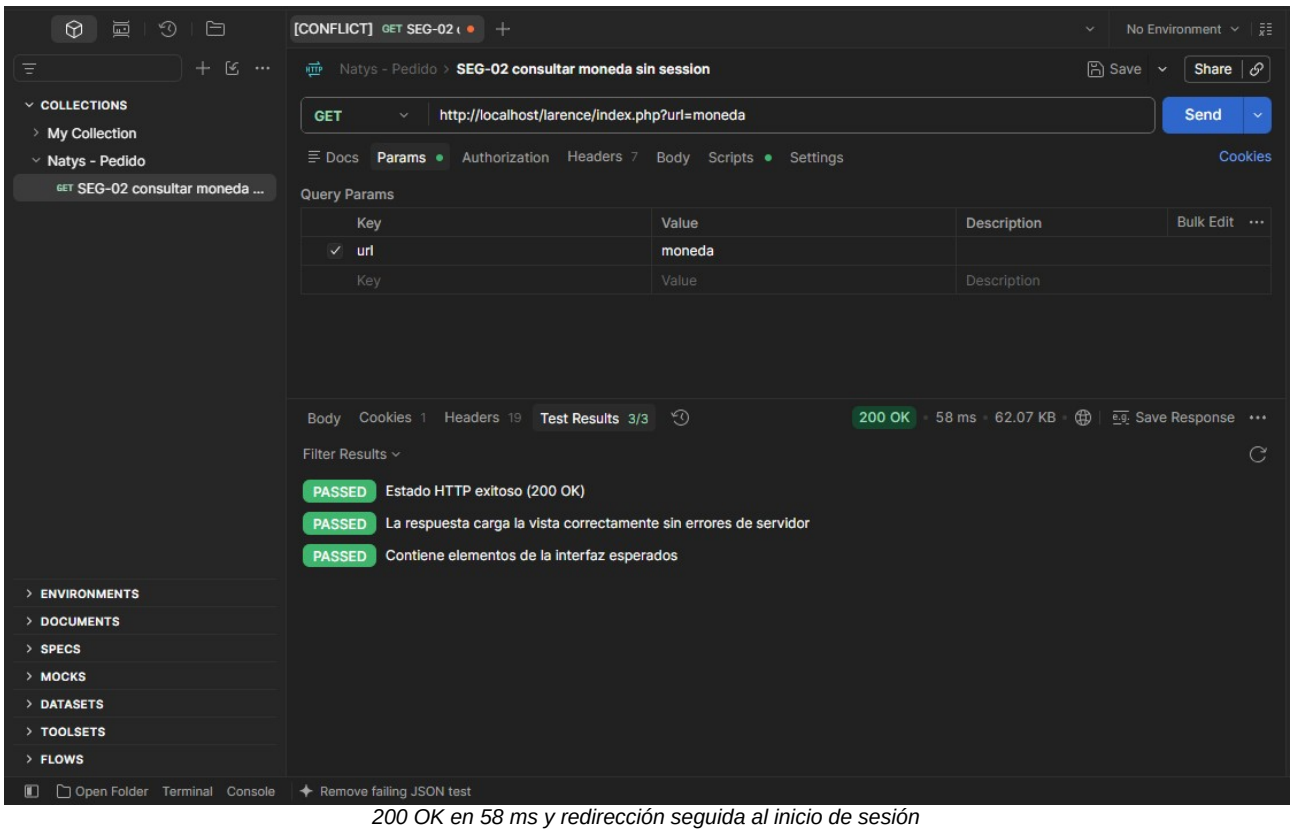
La respuesta final no contiene datos de la moneda

7.2 Prueba de seguridad SEC-02: consultar el módulo de monedas sin sesión

La prueba intenta consultar el módulo sin haber iniciado sesión, para comprobar que la página y sus datos quedan protegidos detrás del inicio de sesión.

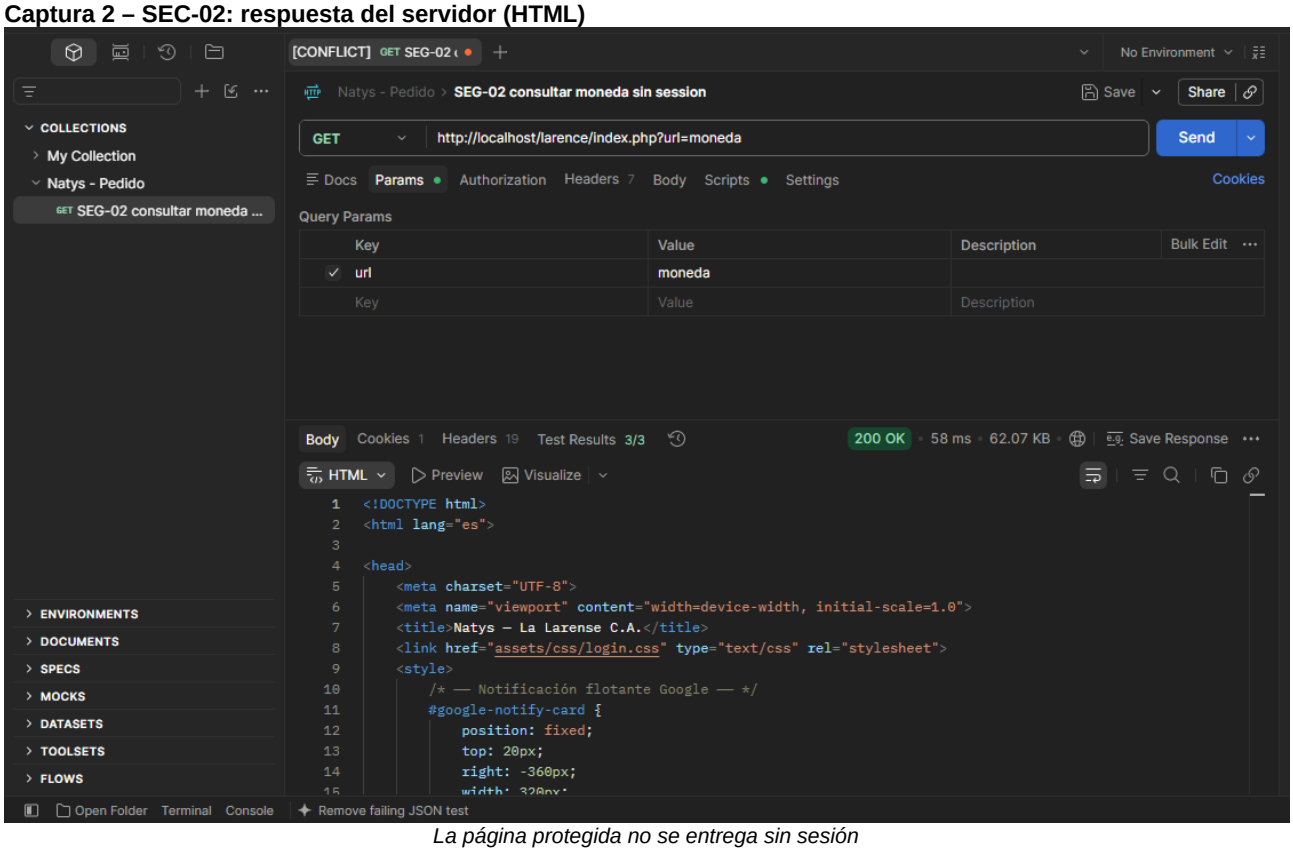
ID	SEC-02
Nombre	Un usuario sin sesión no puede abrir el módulo de monedas
Tipo de prueba	Seguridad (control de acceso)
Herramienta	Postman
Petición	`GET http://localhost/larence/index.php?url=moneda`
Precondiciones	Apache y MySQL activos · petición sin una sesión autenticada · redirecciones seguidas por Postman
Pasos	1. Se duplicó la petición de consulta del módulo y se renombró como SEC-02. 2. Se eliminó la cookie de autenticación de la petición. 3. Se envió la petición GET y se siguió la redirección. 4. Se registraron el código final y la respuesta HTML del servidor.
Resultado esperado	El servidor redirige al inicio de sesión y la respuesta final no entrega la página del módulo.
Resultado obtenido	200 OK en 58 ms tras seguir la redirección. La respuesta final es la página de inicio de sesión y no entrega la página del módulo. Las 3 aserciones de Postman se aprobaron.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – SEC-02: estatus de la petición



200 OK en 58 ms y redirección seguida al inicio de sesión

Captura 2 – SEC-02: respuesta del servidor (HTML)



La página protegida no se entrega sin sesión

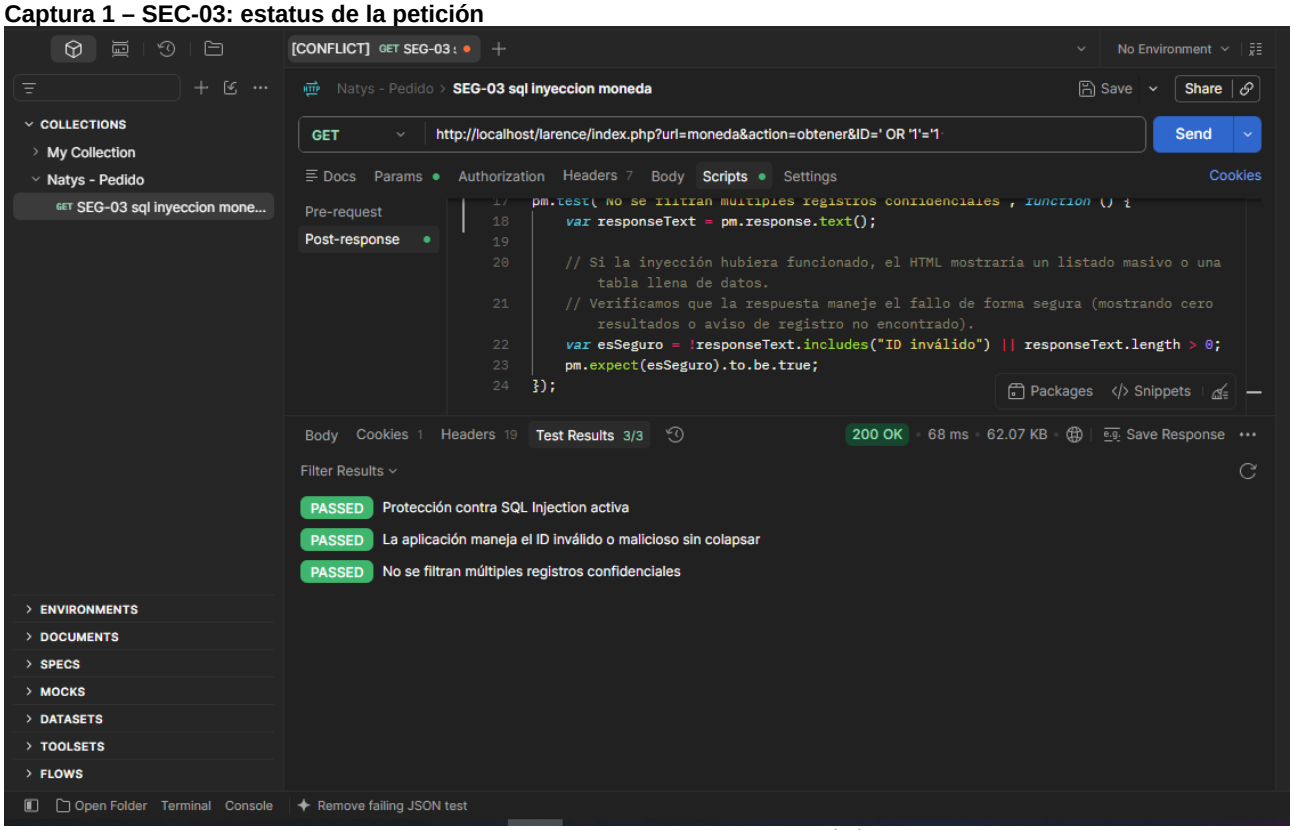
7.3 Prueba de seguridad SEC-03: inyección SQL en el módulo de monedas

La prueba envía una cadena de inyección SQL en el identificador del registro para comprobar que el sistema la trata como un valor y no como código ejecutable.

ID	SEC-03

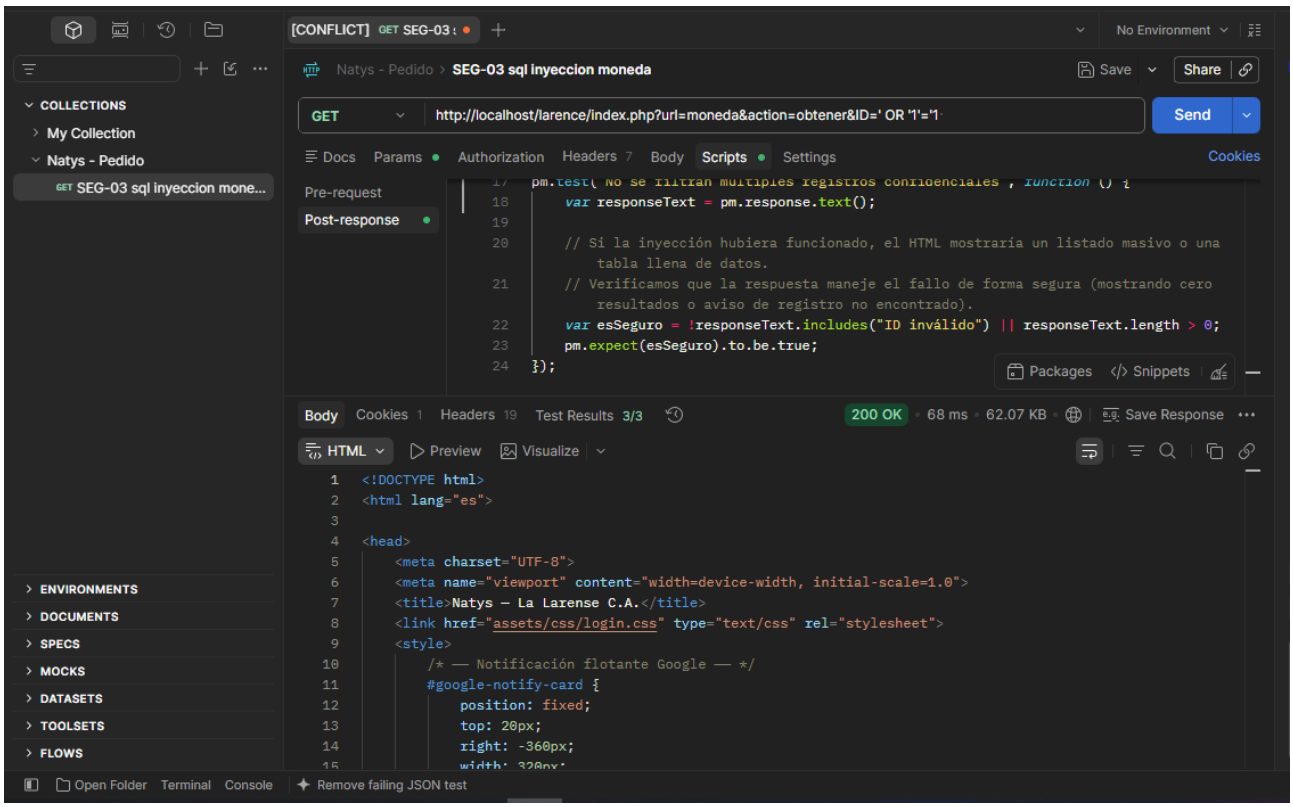
Nombre	El identificador de la moneda resiste la inyección SQL
Tipo de prueba	Seguridad (inyección SQL)
Herramienta	Postman
Petición	`GET http://localhost/larence/index.php?url=moneda&action=obtener&ID=1' OR '1'='1`
Descripción	Envía en el parámetro ID una cadena de inyección SQL clásica. Un sistema vulnerable devolvería un listado masivo de monedas o un error de SQL.
Precondiciones	Apache y MySQL activos · sesión iniciada (cookie de sesión válida en el encabezado).
Datos de entrada	`ID = 1' OR '1'='1`
Pasos	1. Se duplicó la petición de consulta y se renombró como SEC-03. 2. Se reemplazó el valor del parámetro ID por la cadena de inyección. 3. Se envió la petición GET con la sesión iniciada. 4. Se verificó que no apareciera un error SQL ni un listado masivo de registros.
Resultado esperado	Código 200, respuesta HTML, sin mensajes de error SQL y sin un listado masivo de monedas.
Resultado obtenido	200 OK en 68 ms. La respuesta HTML se procesa sin error SQL y no muestra un listado masivo de monedas. Las 3 aserciones de Postman se aprobaron.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – SEC-03: estatus de la petición



200 OK en 68 ms y respuesta HTML controlada

Captura 2 – SEC-03: respuesta del servidor (HTML)



Sin errores de SQL y sin listado masivo de registros

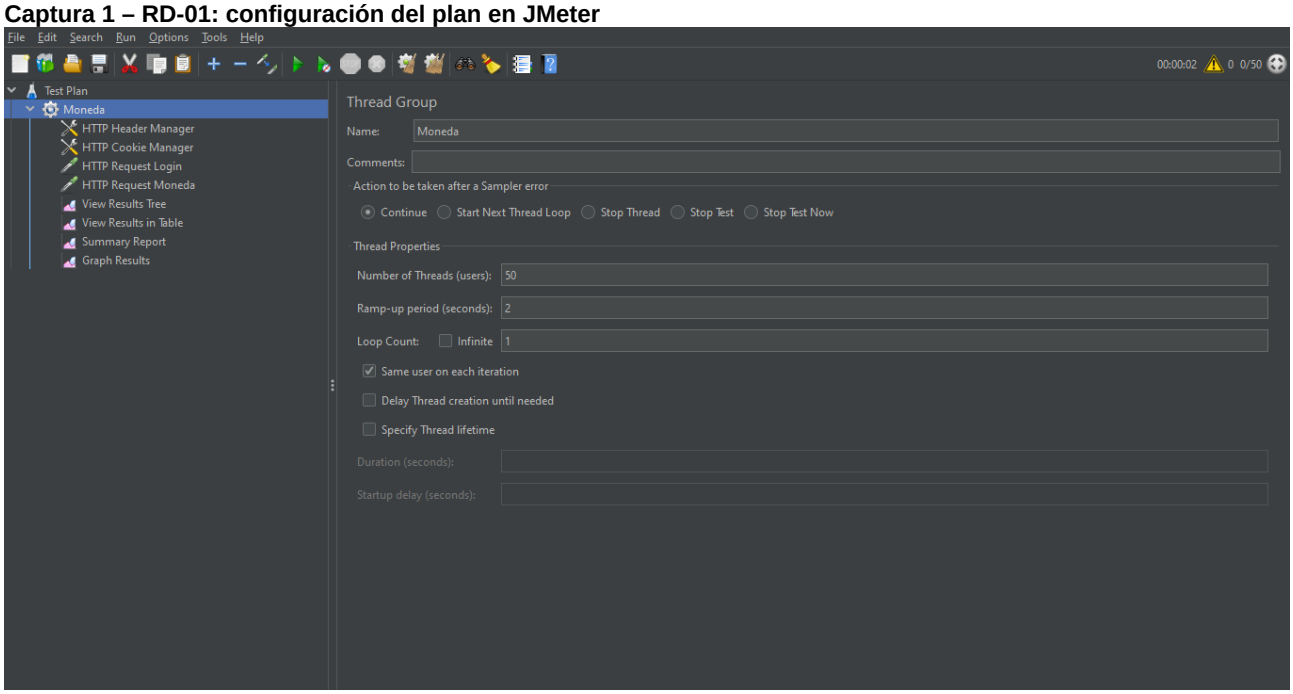
8.1 Prueba de rendimiento RD-01: Moneda

La prueba de rendimiento simula 50 usuarios virtuales que inician sesión y consultan el módulo al mismo tiempo, con el fin de medir el tiempo de respuesta y la estabilidad del sistema bajo carga (RNF 05).

ID	RD-01
Nombre	50 usuarios consultan el listado de monedas al mismo tiempo
Tipo de prueba	Rendimiento / carga (no funcional)
Herramienta	Apache JMeter 5.6.3 · plan de prueba del módulo Moneda
Petición	`POST http://localhost/larence/index.php?url=autenticator&action=ingresar (login) + GET http://localhost/larence/index.php?url=moneda`
Configuración	Thread Group: 50 usuarios virtuales · ramp-up de 2 segundos · Loop Count infinito, detenido al completar 100 solicitudes (50 login + 50 módulo).
Precondiciones	Apache y MySQL activos · usuario de prueba válido · HTTP Cookie Manager con la cookie de sesión.
Pasos	1. Se creó el plan de prueba del módulo en Apache JMeter 5.6.3. 2. Se agregaron HTTP Request Defaults (localhost:80, ruta /larence), HTTP Cookie Manager y el Thread Group con 50 usuarios y ramp-up de 2 segundos. 3. Se agregaron las peticiones HTTP Request Login y HTTP Request Moneda. 4. Se agregó el listener Summary Report y se ejecutó el plan con el Thread Group en modo infinito. 5. Se detuvo la ejecución al completar 100 solicitudes y se registraron las métricas.
Resultado esperado	100 solicitudes procesadas, tiempo promedio menor a 2.000 ms y 0 % de errores.
Resultado obtenido	100 solicitudes (50 login + 50 consultas al módulo), promedio de 56 ms (mínimo 27 ms, máximo 105 ms), desviación estándar de 18,43 ms, 0,00 % de errores y 48,6 solicitudes/s.
Estado	**Aprobado**

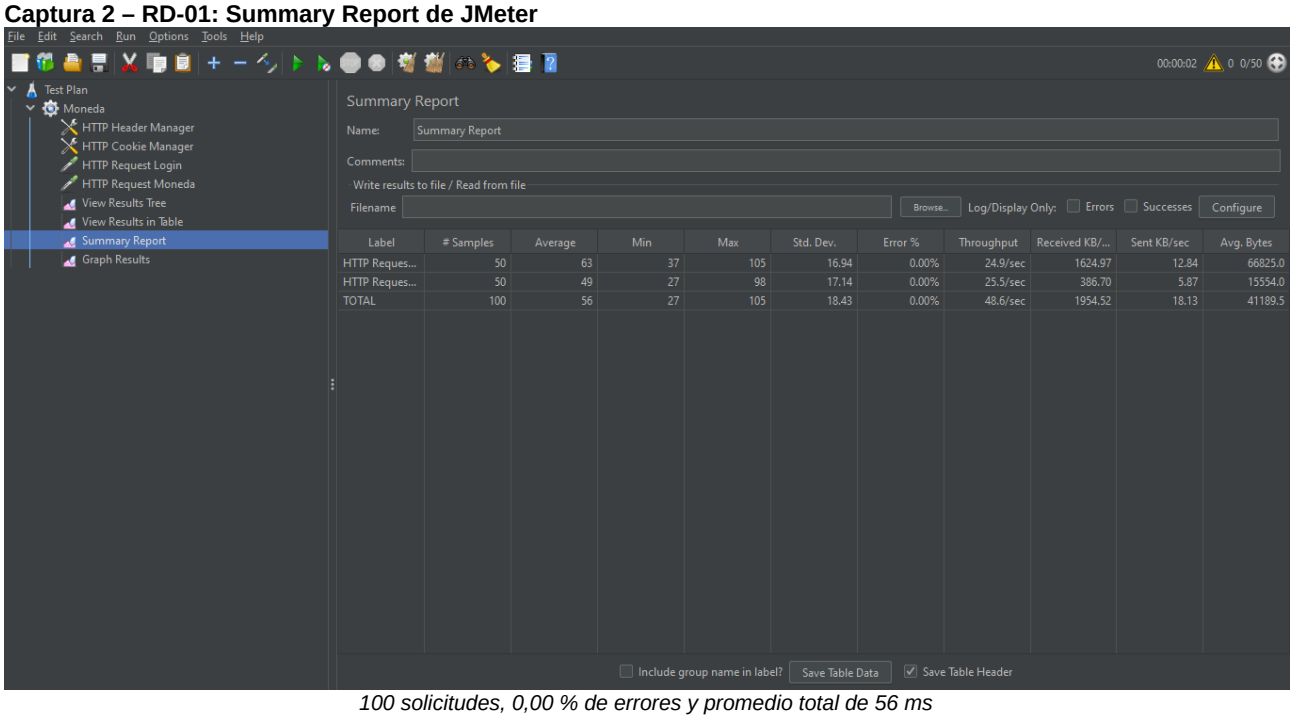
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – RD-01: configuración del plan en JMeter



Thread Group con 50 usuarios y ramp-up de 2 segundos

Captura 2 – RD-01: Summary Report de JMeter



100 solicitudes, 0,00 % de errores y promedio total de 56 ms

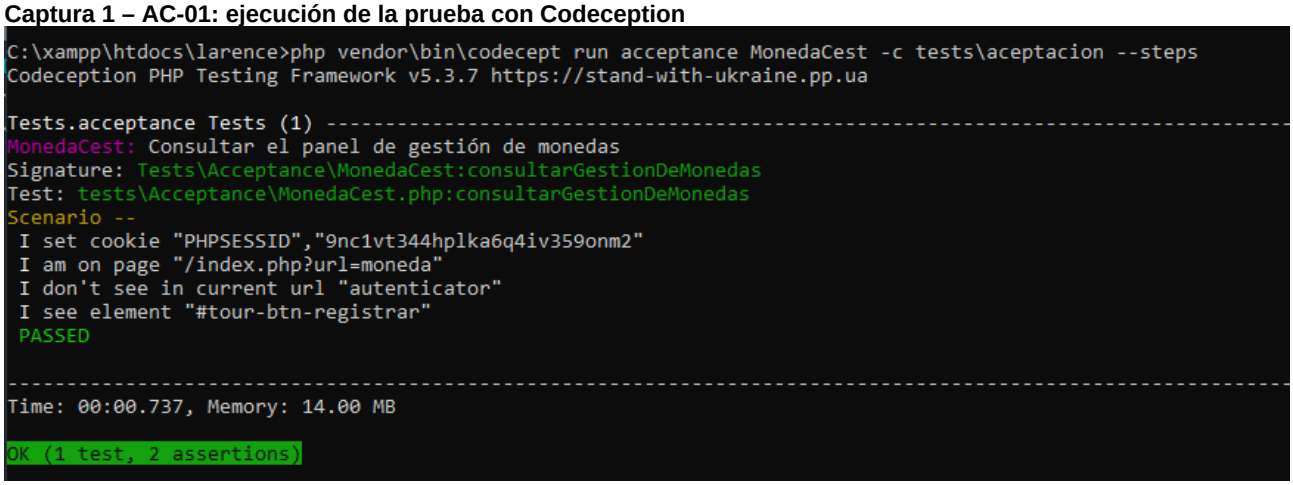
9.1 Prueba de aceptación AC-01: Moneda

La prueba de aceptación verifica, desde el navegador, que el módulo cumple el flujo de consulta solicitado por el cliente. Se escribió como una historia de usuario con Codeception y se ejecutó con PhpBrowser.

ID	AC-01
Nombre	El usuario consulta el panel de monedas
Tipo de prueba	Aceptación
Herramienta	Codeception 5.3.7 con el módulo PhpBrowser
Requisitos cubiertos	CRUD Leer: consulta del panel de monedas y de su interfaz principal.

Archivo y método	tests/aceptacion/tests/Acceptance/MonedaCest.php · consultarGestionDeMonedas
Precondiciones	Apache y MySQL activos · sesión iniciada en el sistema · cookie PHPSESSID válida.
Pasos	1. Se configuró Codeception 5.3.7 con el módulo PhpBrowser y la URL base del sistema. 2. Se fijó la cookie PHPSESSID de una sesión válida. 3. Se abrió GET /index.php?url=moneda con amOnPage y se verificó que no redirigiera a autenticator. 4. Se comprobó la presencia del botón de registro #tour-btn-registrar. 5. Se ejecutó: php vendor\bin\codecept run acceptance MonedaCest -c tests\ aceptacion --steps.
Resultado esperado	El panel de monedas abre sin redirección al inicio de sesión y muestra el botón de registro (#tour-btn-registrar).
Resultado obtenido	Escenario PASSED: el panel de monedas abrió correctamente y el botón de registro estuvo presente. OK (1 prueba, 2 aserciones) en 0,737 s (14 MB).
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – AC-01: ejecución de la prueba con Codeception



Escenario PASSED: el panel de monedas abrió correctamente y el botón de registro estuvo presente. OK (1 prueba, 2 aserciones) en 0,737 s (14 MB).